

Lecture 09

RISC-V: Pipelined Implementation – Data Hazards

Instructor: Dr. Muhammad Imran



Computer Architecture | RISC-V Pipelined Implementation

Acknowledgement

- Content from following has been used in these lectures
 - Computer Organization and Design, RISC-V 2nd Edition, Patterson and Hennessy

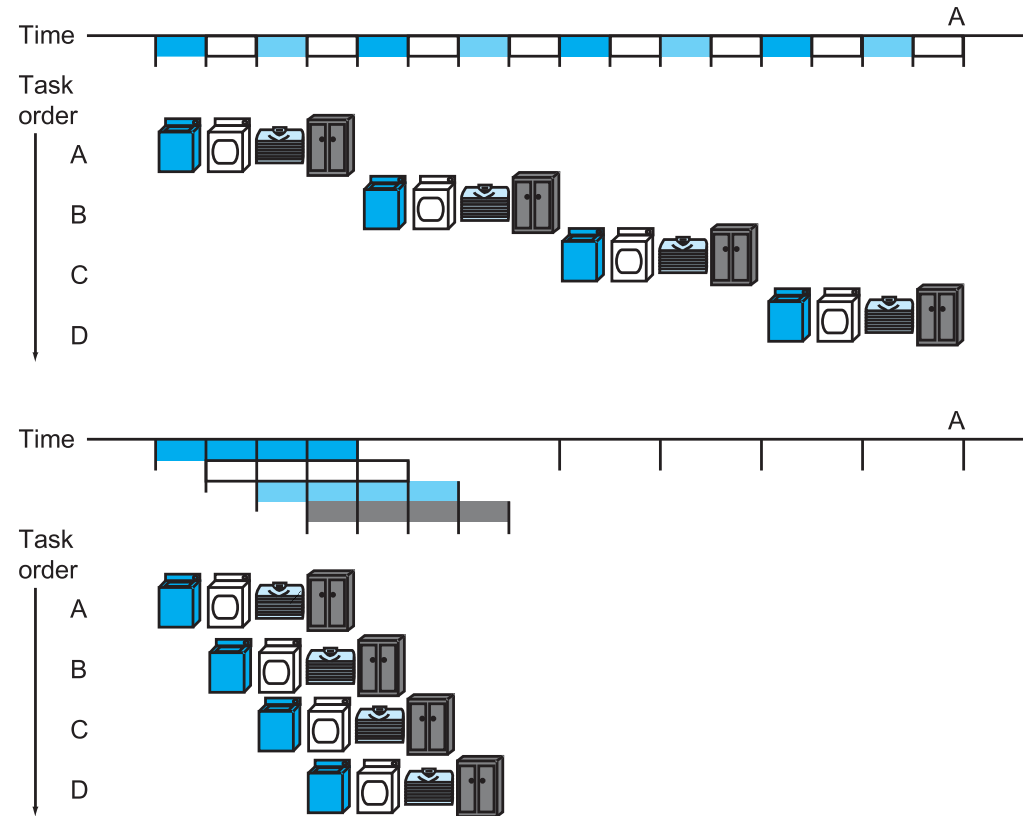
Contents

- Pipelining – An Overview
- Pipelined Microarchitecture
- Dependencies and Pipelining Hazards
- Dealing with Pipelining Hazards in Hardware
- Handling Multicycle Operations

Pipelining to Improve Throughput



A Laundry Analogy



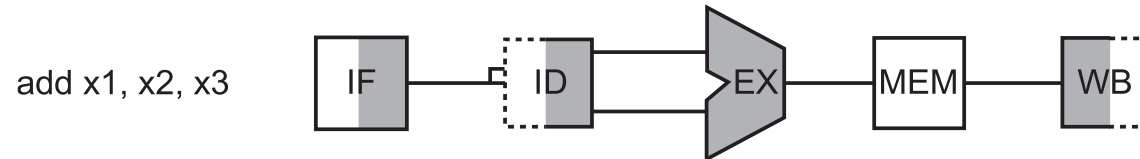
- Pipelining helps execute multiple tasks in parallel
- Improves throughput ...

Improvement by **Pipelining**

- Pipelining only improves throughput
- Individual tasks still take same amount of time
- When number of tasks is too large and pipeline stages are perfectly **balanced!**
 - Improvement in performance $\approx$ Number of pipeline stages
 - Example
 - Suppose each stage in laundry takes 10 minutes and there are 4 stages
 - Without pipelining
 - 4 loads take $40 \times 4 = 160$ minutes!
 - With pipelining
 - 4 loads take 70 minutes!
 - With large number of loads, improvement would approach 4 times!

Pipelining in RISC-V

- Instruction execution stages
 1. Fetch instruction from memory.
 2. Read registers and decode the instruction.
 3. Execute the operation or calculate an address.
 4. Access an operand in data memory (if necessary).
 5. Write the result into a register (if necessary).
- Therefore, we can implement a **five-stage** pipeline for RISC-V!



- RISC-V Pipeline Stages are not perfectly balanced!

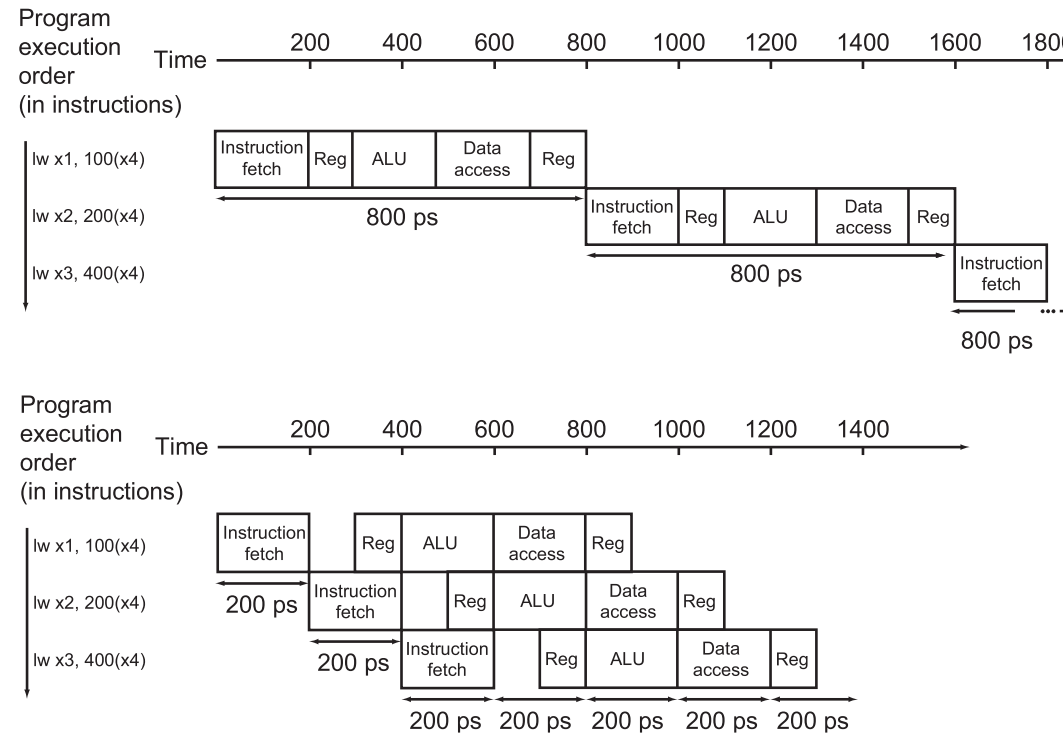
Pipelining in RISC-V

- Example of RISC-V instruction execution time!

Instruction class	Instruction fetch	Register read	ALU operation	Data access	Register write	Total time
Load word (lw)	200 ps	100 ps	200 ps	200 ps	100 ps	800 ps
Store word (sw)	200 ps	100 ps	200 ps	200 ps		700 ps
R-format (add, sub, and, or)	200 ps	100 ps	200 ps		100 ps	600 ps
Branch (beq)	200 ps	100 ps	200 ps			500 ps

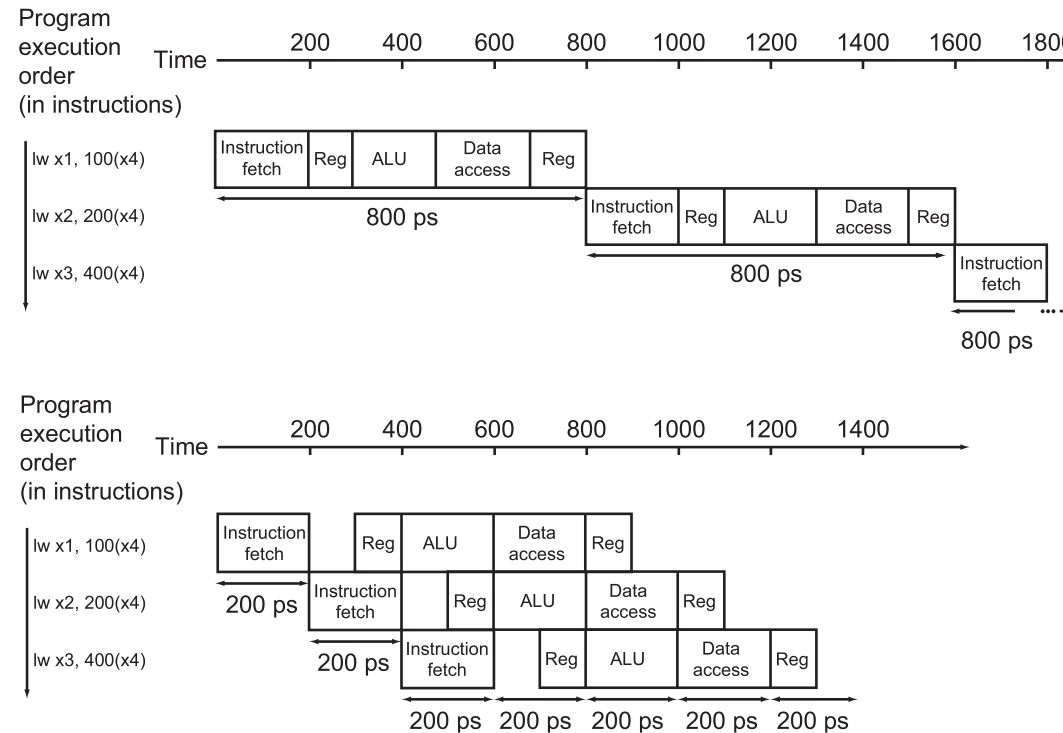
- Assuming Multiplexors, Control Unit, PC Access and Sign-extension unit has no delay!
- Load takes the longest!
 - For single-cycle, the clock period must be that of load time!
- Among individual stages the longest time is 200 ps!
 - The clock period in pipelined architecture must be at least 200 ps (+ t_{clk2q} + t_s), although some stages take only 100 ps!!

Pipelining in RISC-V



- Time to execute three instructions
 - Without pipelining: 2400 ps
 - After pipelining: 1400 ps
 - Time between first and 3rd instruction = $2 \times 200 \text{ ps} = 400 \text{ ps}$

Pipelining in RISC-V



- How about adding 1,000,000 more instructions (to current 3)?
 - Each instruction adds 200 ps, total execution time using pipelining would be $(1400 \text{ ps} + 200 \times 1,000,000 \text{ ps}) = 200,001,400 \text{ ps}$
 - For non-pipelined design execution time = $(2400 \text{ ps} + 1,000,000 \times 800 \text{ ps}) = 800,002,400 \text{ ps}$
 - Improved is about 4 times = **amount of reduction in clock period (given imbalanced stages)!**

Pipelining in RISC-V

- Implementing RISC-V Pipelining
 - RISC-V instructions are easier to pipeline, because of
 - Fixed instruction size
 - Easy to fetch and decode!
 - Fixed location of source/destination operands
 - Memory operands appear only in loads/stores
 - Execution stage can be used for address calculation!
 - Access memory in last stage!
 - Allowing memory operand in other instructions would increase number of pipeline stages / imbalance!
 - RISC-V has fewer instruction formats!
 - x86 architecture has variable instruction sizes
 - Hard to pipeline
 - x86 instructions are translated into RISC like operations to implement pipelining

**Pipelining is a programmer invisible
technique for performance
improvement ...**

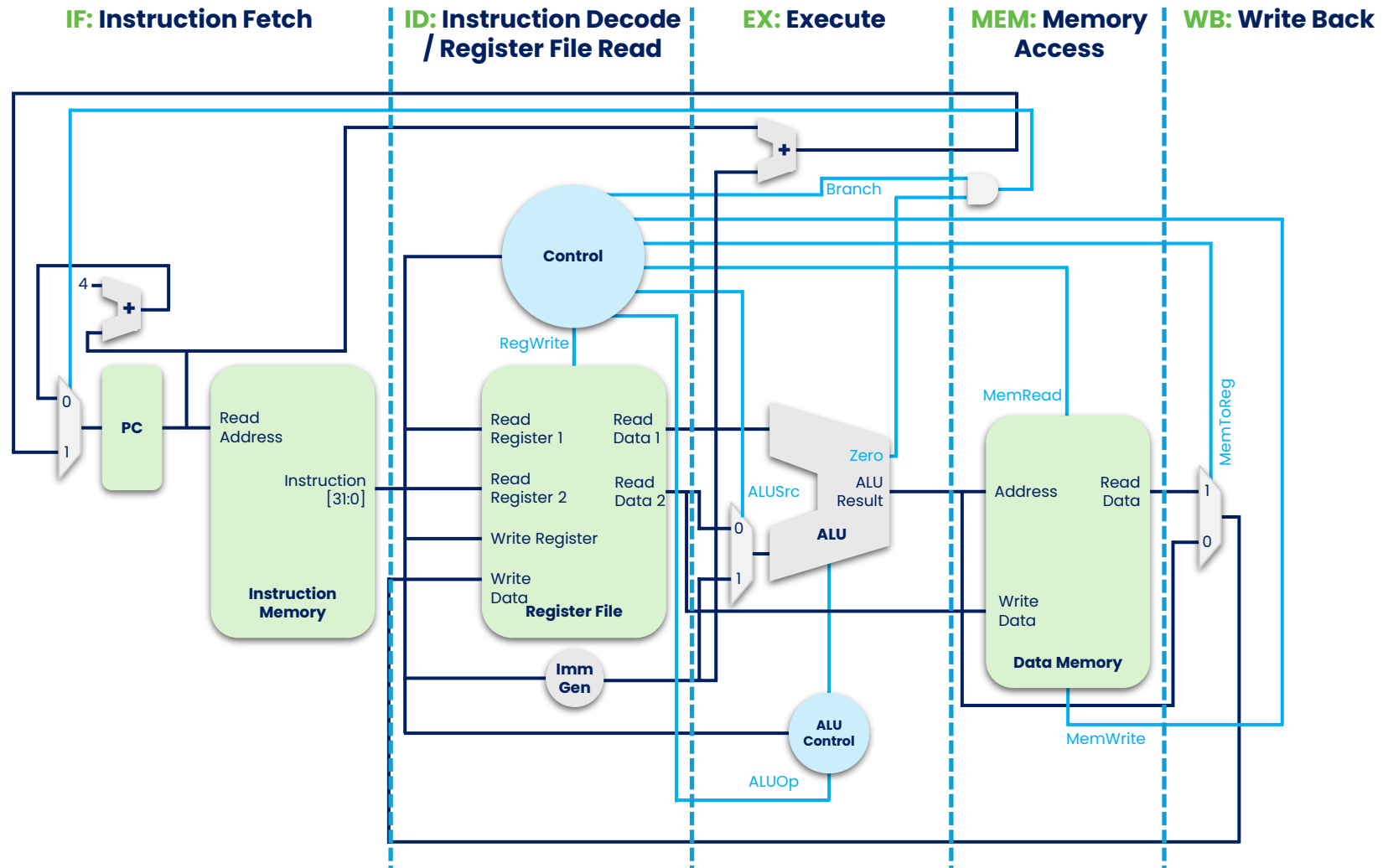
Pipelined Microarchitecture



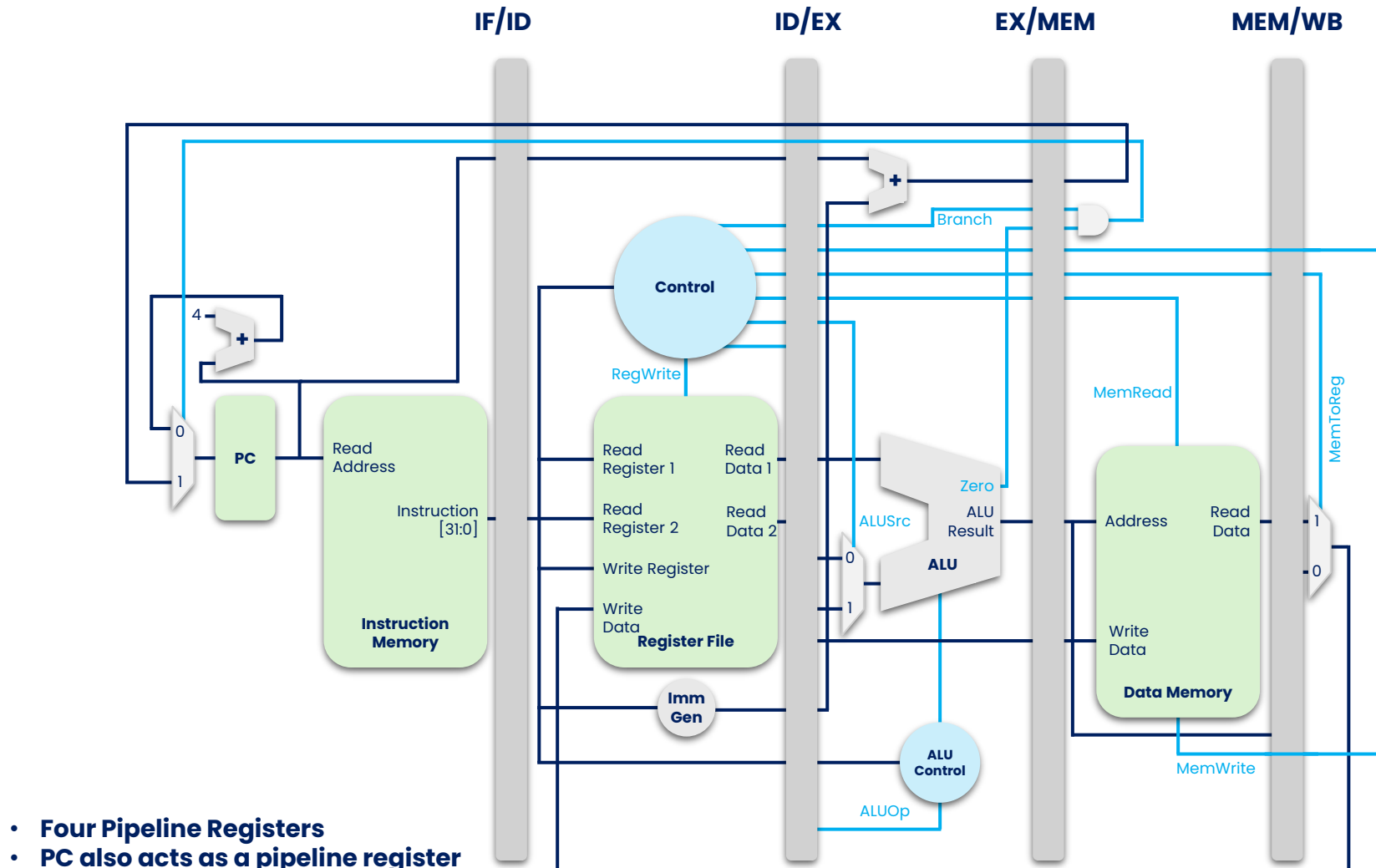
RISC-V Pipeline Stages

1. IF: Instruction fetch
2. ID: Instruction decode and register file read
3. EX: Execution or address calculation
4. MEM: Data memory access
5. WB: Write back

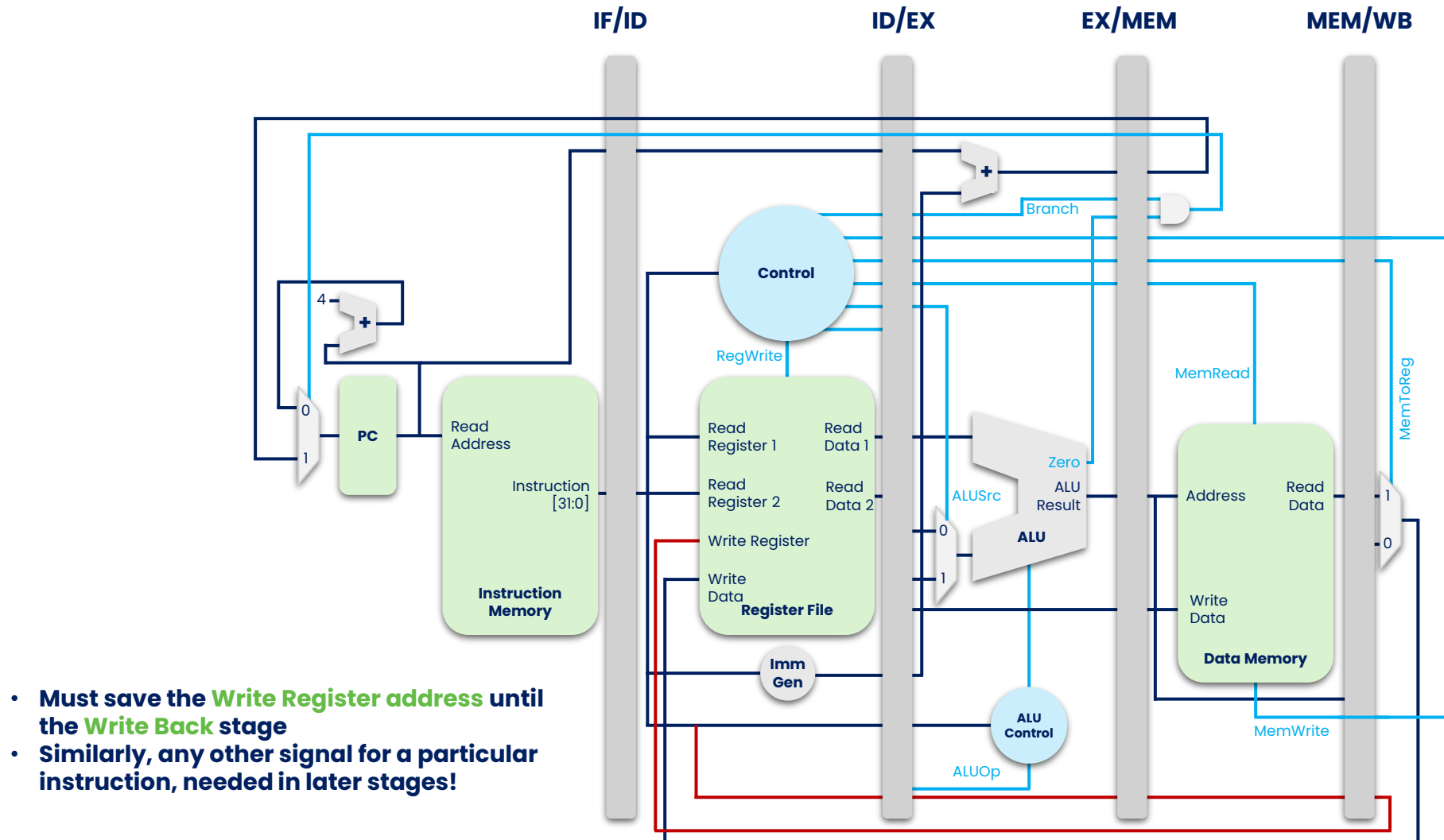
RISC-V Pipeline Stages



Adding Pipeline Registers



Writing Back the **Correct Register**



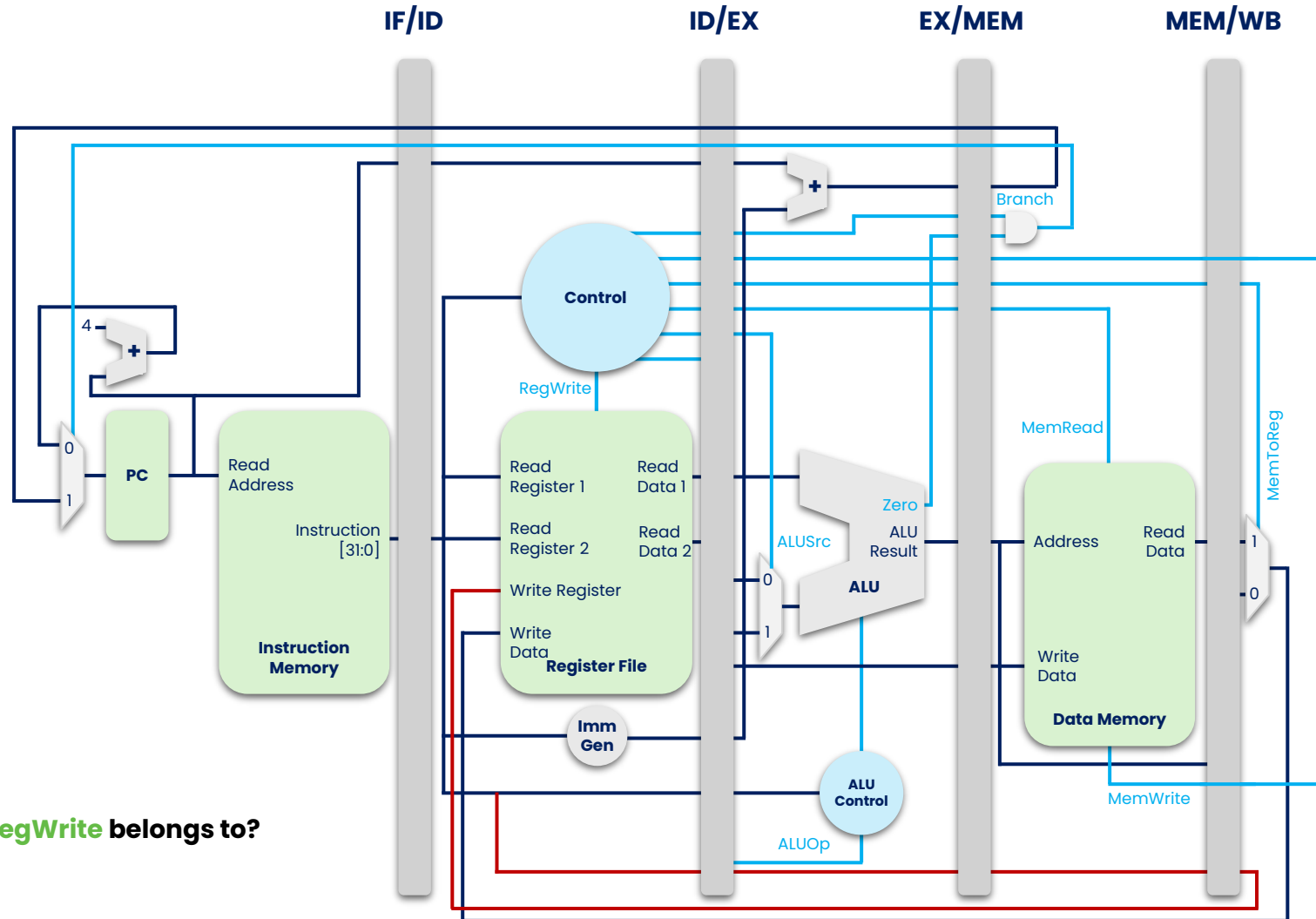
- Must save the **Write Register address** until the **Write Back** stage
- Similarly, any other signal for a particular instruction, needed in later stages!

Key Points!

- For a store instruction, the contents of the register read in Stage 2 must be saved in pipeline
 - Used in 4th Stage
- A store instruction finishes in 4th stage (memory access)
 - Can we execute instructions faster because of this?
 - No, because other instructions are already in process!
 - An instruction must pass through all stages even if it doesn't use some of the stages!
- Each logical component can be used only within a *single* pipeline stage!
 - Otherwise, there would be structural hazard!

Pipelining Control

Control Signals in Different Stages



- Which stage **RegWrite** belongs to?

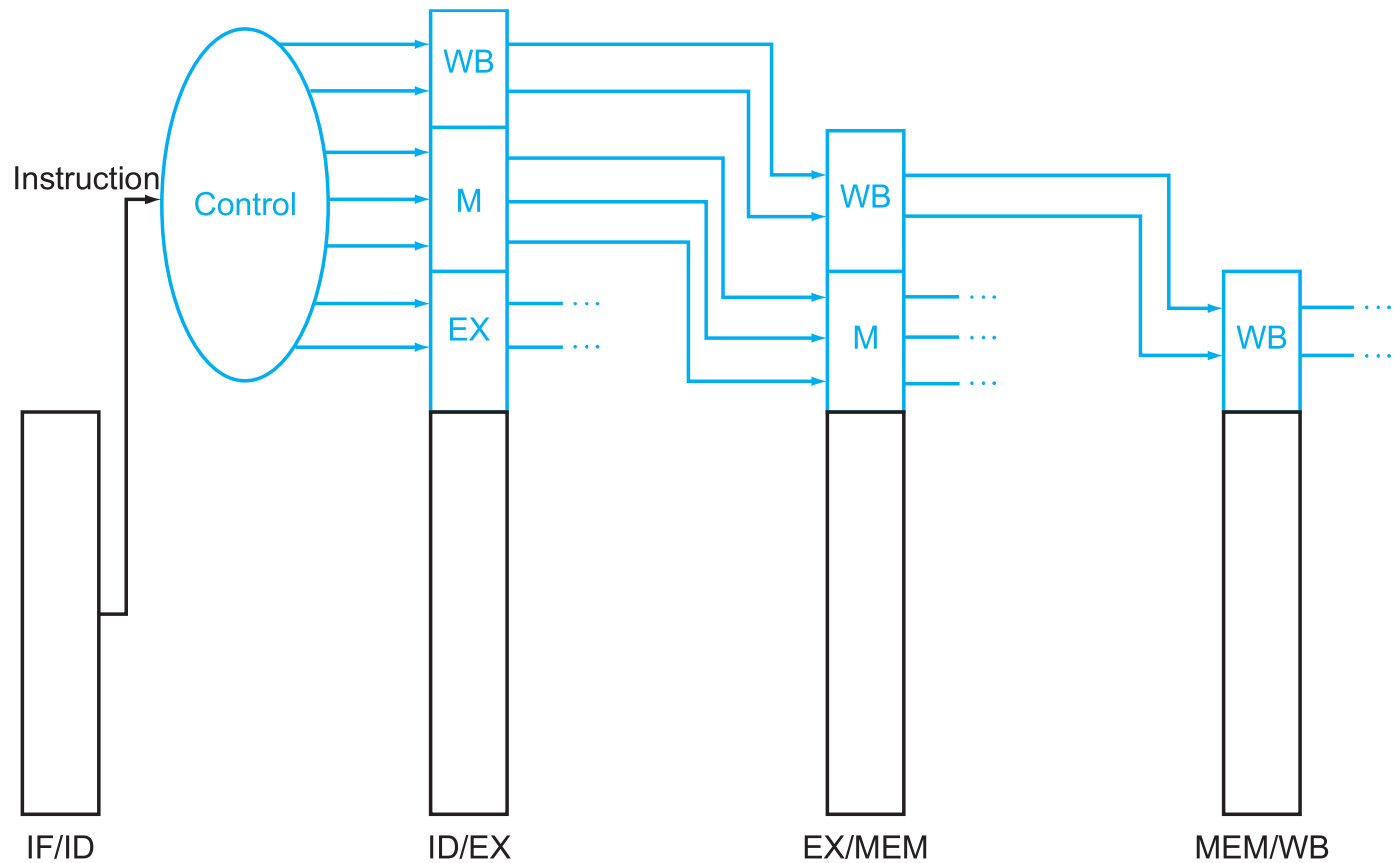
Key in **Pipelining Control**

- Control Signals are generated in Instruction Decode State
- Save control signals in pipeline registers until they are used (in later stages)!

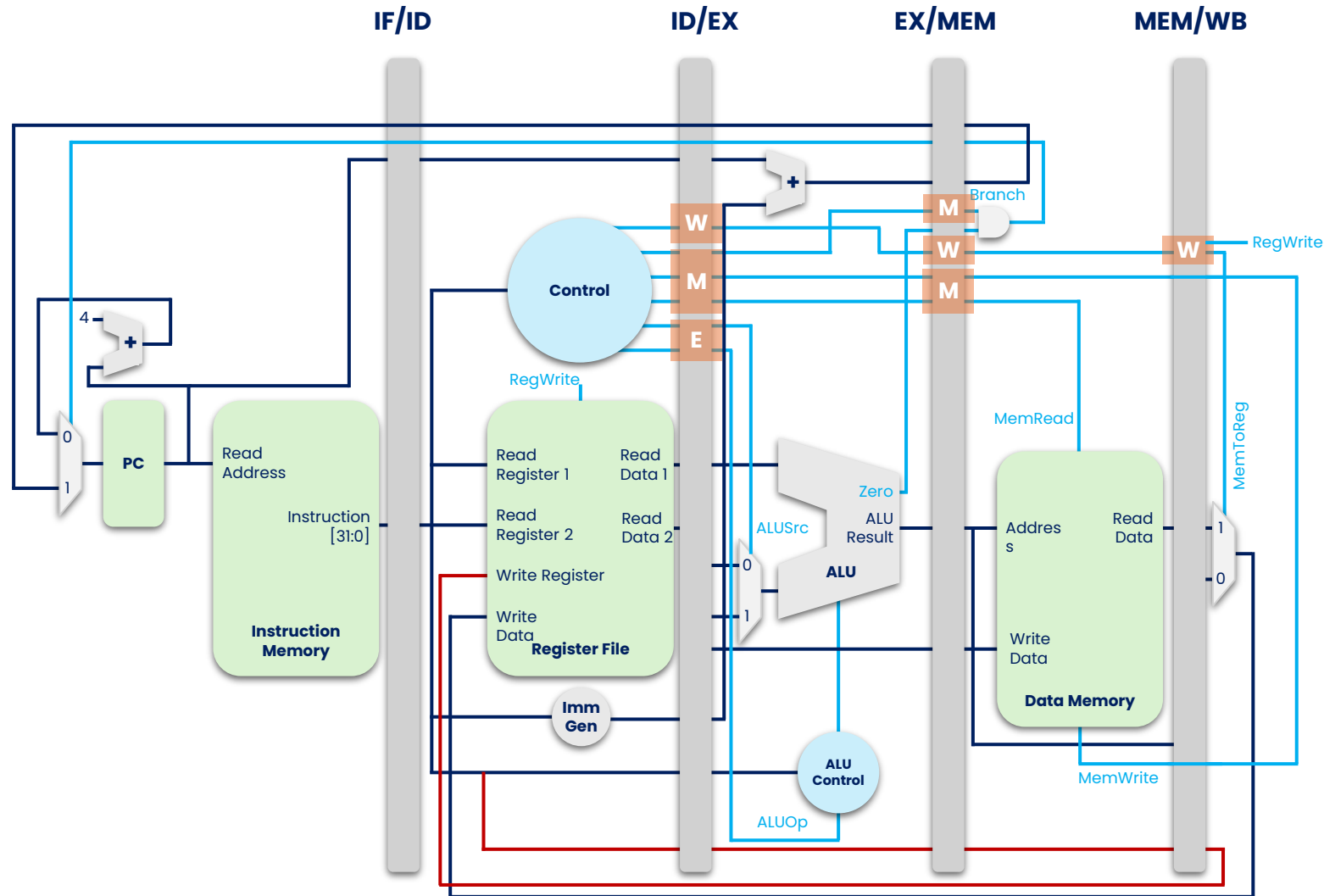
Identifying Control Signals for Each Stage

- Needed in 3 subsequent stages after generation in Second Stage!

Instruction	Execution/address calculation stage control lines		Memory access stage control lines			Write-back stage control lines	
	ALUOp	ALUSrc	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R-format	10	0	0	0	0	1	0
lw	00	1	0	1	0	1	1
sw	00	1	0	0	1	0	X
beq	01	0	1	0	0	0	X



Pipelined Datapath and Control



Dependencies and Pipelining Hazards



Dependences and Hazards

- Dependency
 - An instruction's execution dependent on outcome of another instruction
 - Types of dependences
 - Flow dependences or true data dependences
 - Name dependences
 - Control dependences
- Hazard
 - Situations when an instruction in the pipeline cannot correctly execute in the following cycle
 - Types of hazards
 - Structural Hazards
 - Data Hazards
 - Control Hazards

Dependencies and Hazards

- Dependencies are a property of program
- A dependence may or may not result in hazard!
 - A hazard may or may not require pipeline stall to resolve!
 - This is a property of the pipeline organization!
 - Implementation dependent!

Data Dependences

- Flow dependence
 - $x3 \leftarrow x1 \text{ op } x2$
 - $x5 \leftarrow x3 \text{ op } x4$

**Read after Write
(RAW)**

True Data Dependency

- Anti dependence
 - $x3 \leftarrow x1 \text{ op } x2$
 - $x1 \leftarrow x4 \text{ op } x5$

**Write after Read
(WAR)**

- Output dependence
 - $x3 \leftarrow x1 \text{ op } x2$
 - $x5 \leftarrow x3 \text{ op } x4$
 - $x3 \leftarrow x6 \text{ op } x7$

**Write after Write
(WAW)**

Name Dependency

Do not lead to a hazard in simple in-order pipeline we are discussing now!

Hazards due to name dependences can be addressed by **register renaming!**

Data Dependences

- Data value may flow through registers or memory
 - Data-dependence through registers is easier to detect
 - That through memory is harder
 - 100(x4) and 20(x6) may be same locations!
 - 20(x4) and 20(x4) may be different in different instructions!
 - Data hazard through memory is present when memory is accessed out of program order!
 - Not a case in simple pipeline!

Control Dependences

- Every instruction except that within first basic block is dependent on outcome of a branch!
- Example
 - add x1, x2, x3
 - beq x4, x0, L
 - sub x1, x5, x6
 - ...
 - L:
 - or x7, x1, x8

Structural Hazards

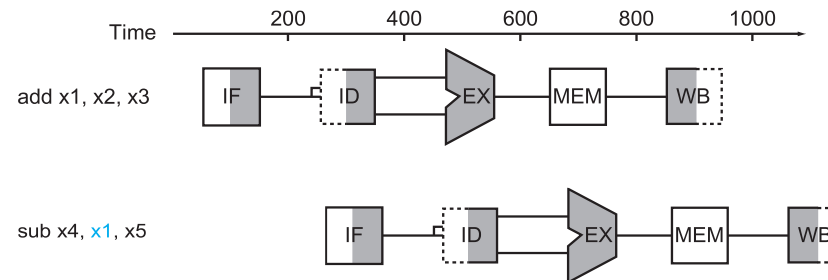
- When hardware cannot execute a combination of instructions in same cycle
 - The required hardware resource is busy!
- Example
 - If we had one memory for instructions and data in RISC-V
 - Memory access from one instruction and instruction fetch of another couldn't execute in one cycle
- RISC-V instruction set was designed to be pipelined
 - Easier to avoid structural hazards in pipelined implementation

Addressing Data Hazards in the Pipeline

In a simple five stage in-order integer pipeline, we only need to address Read-after-Write (RAW) data hazard which is due to true data dependency!

Data Hazards

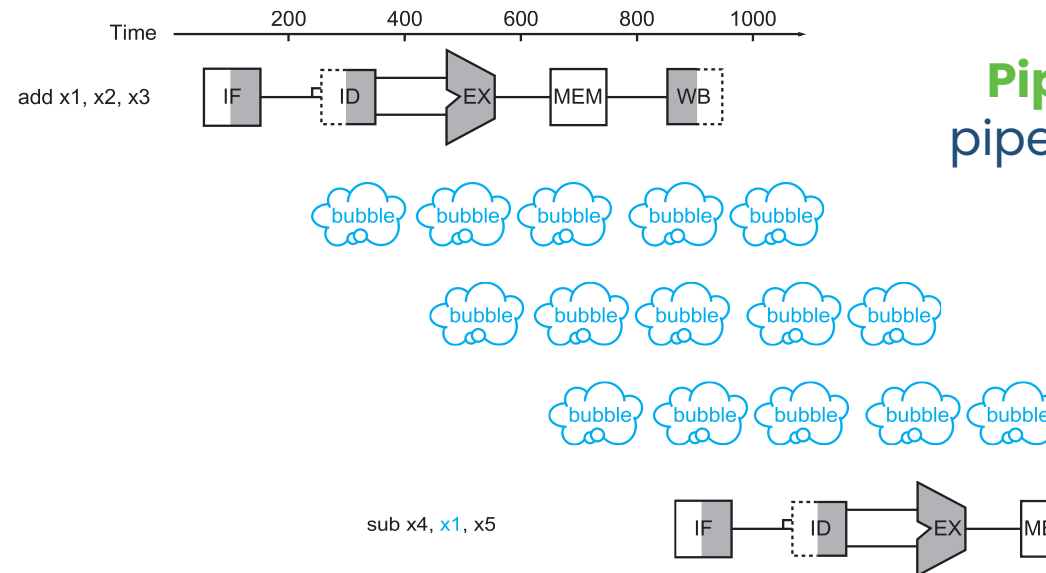
- When instruction cannot execute because the data required by it is not yet available
 - Dependence of one instruction on an earlier instruction in the pipeline
- Example
 - add **x1**, x2, x3
 - sub x4, **x1**, x5
 - x1 will be written back in first instruction later before it is required by 2nd instruction!



Without addressing data hazard outdated value of x1 will be read!

Addressing Data Hazards

- Simple Solution
 - Stall the pipeline!
 - Wait until the data has been written

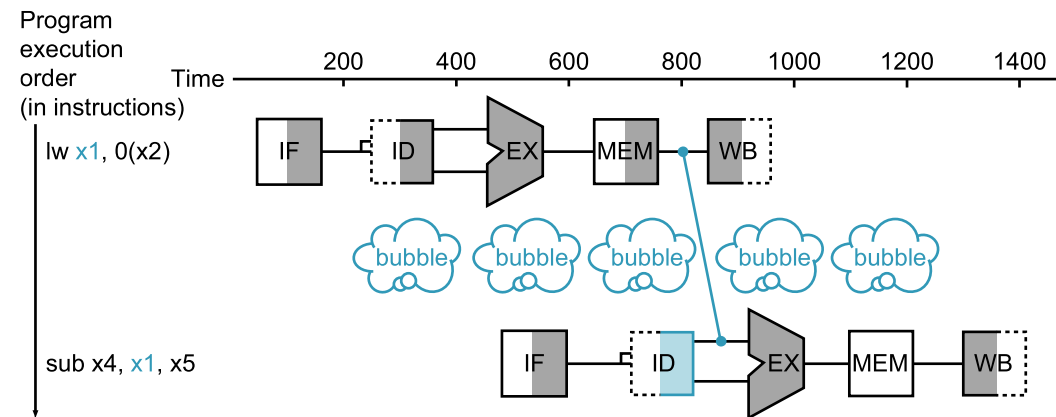


Pipeline Stall or Bubble – A pipeline stall (wait) initiated to resolve a hazard

- Need to wait 3 cycles, so that **slows down** execution!
- Implementing stall
 - Compiler can implement stalls (by adding NOPs) to resolve data hazards!
 - Or hardware can stall the pipeline!

Addressing Data Hazards

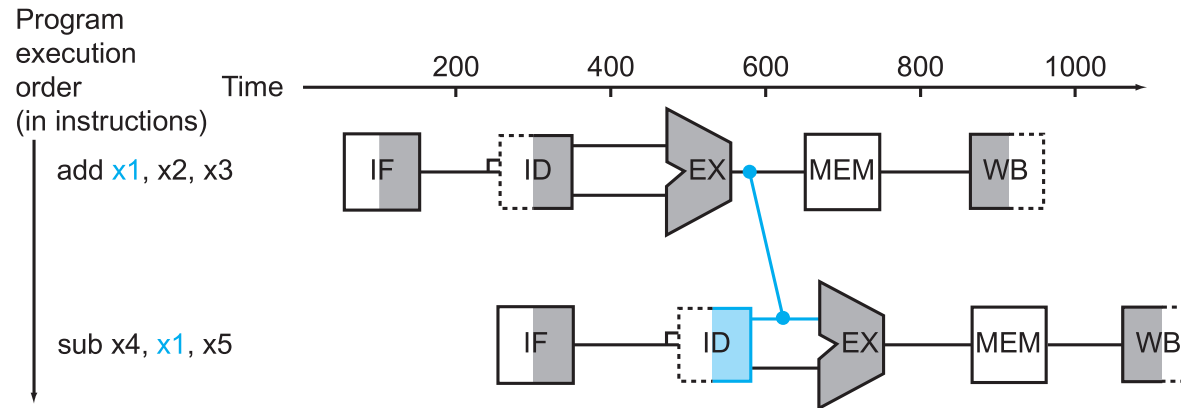
- Forwarding cannot prevent all pipeline stalls!
 - Example
 - lw **x1**, 0(x2)
 - sub x4, **x1**, x5
 - x1 is not available for forwarding when it is needed by 2nd instruction!



- Known as load-use data hazard!

Addressing Data Hazards

- Forwarding or Bypassing
 - Forward the data to the next instruction when it is available
 - Do not wait for the **write back stage** to complete!



- Forwarding only works destination stage is later in time compared to source stage!

Addressing Data Hazards

- Different cases for forwarding!
 - An R-type / load / store instruction dependent on R-type instruction
 - add **x4**, x5, x6
 - sub x7, **x4**, x8
 - Can be resolved by forwarding **result of third stage** of source to **input of third stage** of the destination!
 - An R-type / load / store (for address operand) instruction dependent on load instruction
 - lw **x4**, x5, x6
 - sw x7, 0(**x4**)
 - Can be resolved by **adding a stall** and then forwarding **result of fourth stage** of source to **input of third stage** of the destination!
 - A store (for data operand) instruction dependent on load instruction!
 - lw **x4**, 0(x6)
 - sw **x4**, 0(x8)
 - Can be resolved by forwarding **result of fourth stage** of source to **input of fourth stage** of the destination! (**without any stall !!!**)

Addressing Data Hazards

- Different cases for forwarding!
 - From 3rd or 4th stage to 3rd stage!
 - From 4th stage to 4th stage!
- To resolve data hazards in hardware
 - Need to implement above two cases of forwarding!
 - And
 - Mechanism to stall pipeline for one cycle when needed!
 - An R-type / load / store (for address operand) instruction dependent on load instruction!

Addressing Data Hazards

- Code reordering / static scheduling to prevent stalls
 - Example
 - Consider following C Code
 - $a = b + e;$
 - $c = b + f;$
 - Compiled to following RISC-V Code
 - `lw x1, 0(x31)` # Load b
 - `lw x2, 8(x31)` # Load e
 - `add x3, x1, x2` # b + e
 - `sw x3, 24(x31)` # Store a
 - `lw x4, 16(x31)` # Load f
 - `add x5, x1, x4` # b + f
 - `sw x5, 32(x31)` # Store c
 - Which instructions have data hazards?
 - How many hazards are there **with** or **without** forwarding?

Addressing Data Hazards

- Code reordering to prevent stalls
 - Example
 - Data hazards when we can use forwarding

lw x1, 0(x31)	lw x1, 0(x31)
lw x2, 8(x31)	lw x2, 8(x31)
add x3, x1, x2	lw x4, 16(x31)
sw x3, 24(x31)	add x3, x1, x2
lw x4, 16(x31)	sw x3, 24(x31)
add x5, x1, x4	add x5, x1, x4
sw x5, 32(x31)	sw x5, 32(x31)



- How can we reorder code to avoid stalls?
- Simply moving **lw instruction** above in the sequence can avoid both hazards!
- How much this improves the performance?
 - The new code sequence executes two cycles faster (assuming that we are using forwarding!)

Addressing Data Hazards

- Forwarding implementation in RISC-V
 - RISC-V instructions write at most one result!
 - Result is written in last stage!
- Forwarding is harder if
 - More than one results are to be forwarded per instruction!

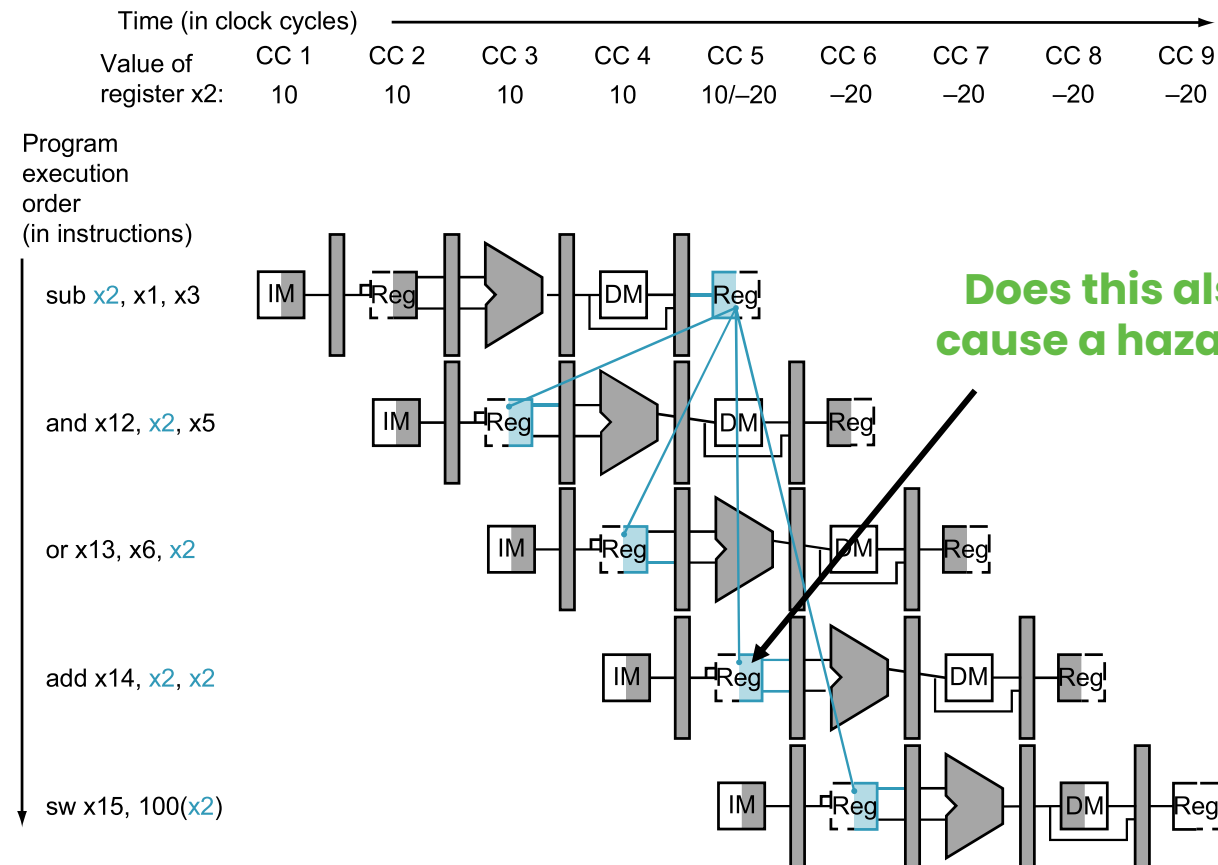
Implementing forwarding to address data hazards!

Revisiting Data Hazards

- Example of data dependencies
 - sub **x2**, x1, x3 # Register x2 written by sub
 - and x12, **x2**, x5 # 1st operand(x2) depends on sub
 - or x13, x6, **x2** # 2nd operand(x2) depends on sub
 - add x14, **x2**, **x2** # 1st(x2) & 2nd(x2) depend on sub
 - sw x15, 100(**x2**) # Base (x2) depends on sub
- Which instructions can have data hazards?

Revisiting Data Hazards

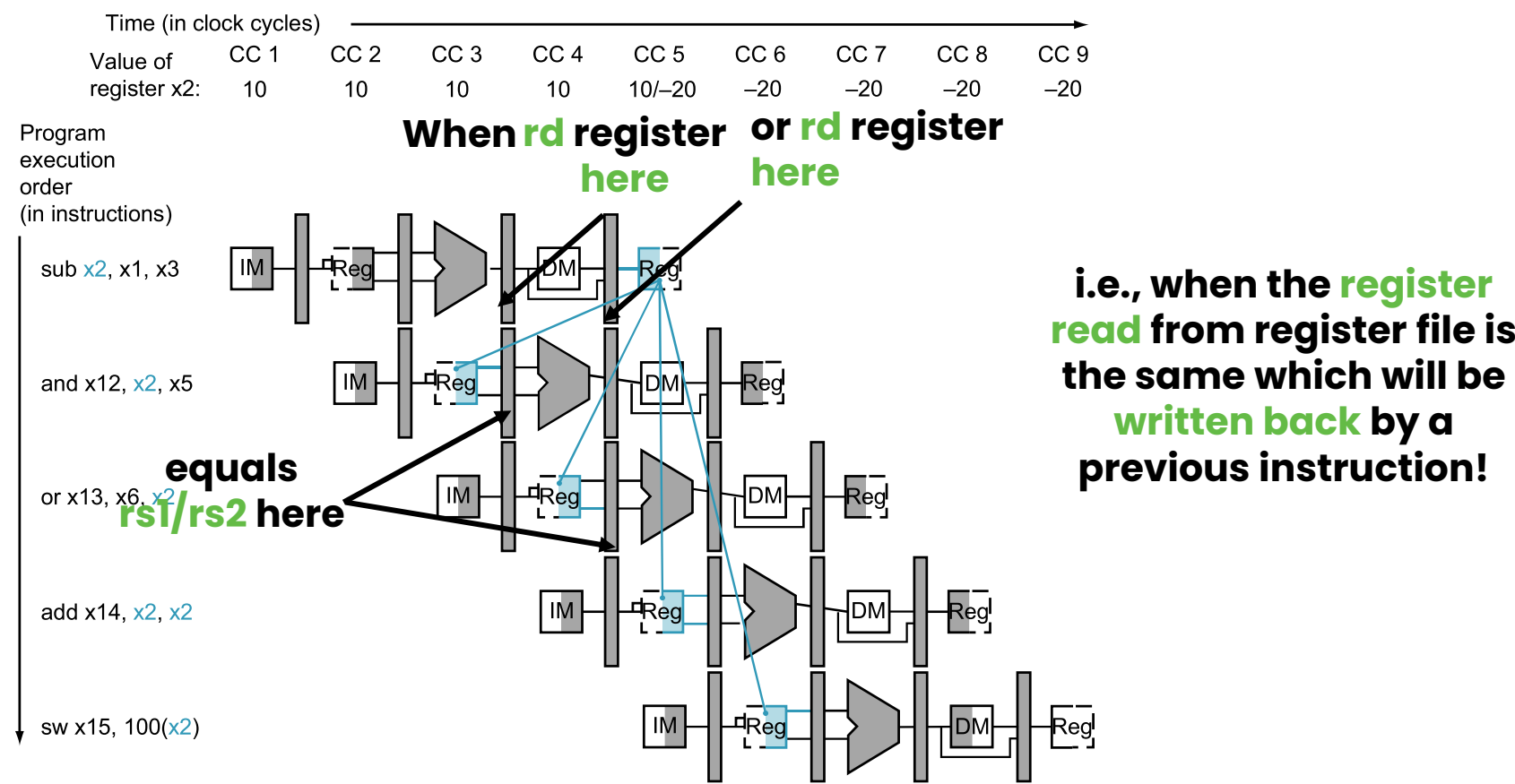
- Example of data hazards



Yes, it is, but preventable within the register file!

Detecting Data Hazards

- How can we **detect** data hazards in hardware?



Detecting Data Hazards

- Conditions for Hazards

1a. EX/MEM.RegisterRd = ID/EX.RegisterRs1

1b. EX/MEM.RegisterRd = ID/EX.RegisterRs2

2a. MEM/WB.RegisterRd = ID/EX.RegisterRs1

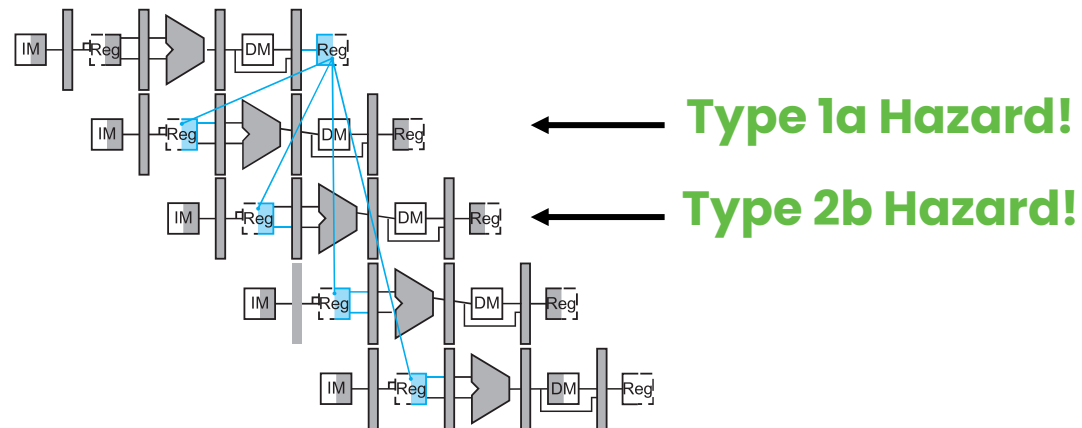
2b. MEM/WB.RegisterRd = ID/EX.RegisterRs2

EX Hazard!

MEM Hazard!

- Example

- sub **x2**, x1, x3
- and x12, **x2**, x5
- or x13, x6, **x2**
- add x14, **x2**, **x2**
- sw x15, 100(**x2**)



Is there any problem with this Hazard Detection Policy?

Detecting Data Hazards

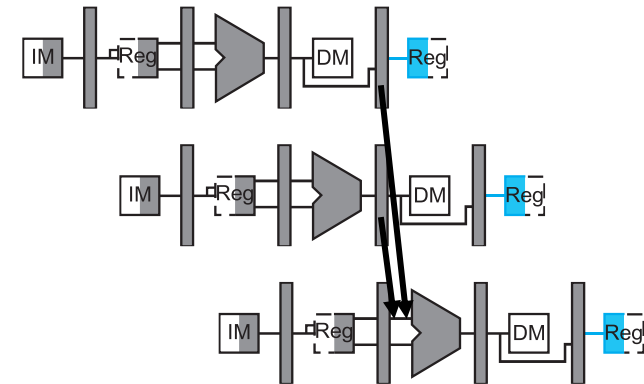
- Rd of an instruction (in EX or MEM Stage) can be equal to rs1/rs2 of a subsequent instruction even though this instruction doesn't write back!
 - Using forwarding for this case would be wrong!
- Solution?
 - Check if **RegWrite** is active!
 - Which is in the **WB Control Signals** of EX/MEM register!
- Any other problems with Hazard Detection Policy?
 - When x0 is the destination!!
 - Wrong non-zero result would be forwarded!
 - x0 as a source operand in subsequent instruction would get non-zero forwarded data!
 - So, do not forward when rd = x0 !!!
- Any other problem ??? ... 😊

Detecting Data Hazards

- Final Problem
 - What happens if two hazard detection conditions (EX Hazard and MEM Hazard) occur at the same time?
 - $EX/MEM.RegisterRd = ID/EX.RegisterRs1/Rs2$
 - $MEM/WB.RegisterRd = ID/EX.RegisterRs1/Rs2$
 - Example
 - Summing a vector in a register
 - add x1, x1, x2
 - add x1, x1, x3
 - add x1, x1, x4
 - ...
 - Two possible sources of forwarding
 - From Memory Access Stage (MEM Hazard)!
 - From Execute Stage (EX Hazard)!
 - The updated data is in EX Stage!
 - Thus, we need to discard **MEM Hazard** in this case!

EX Hazard →

EX + MEM →

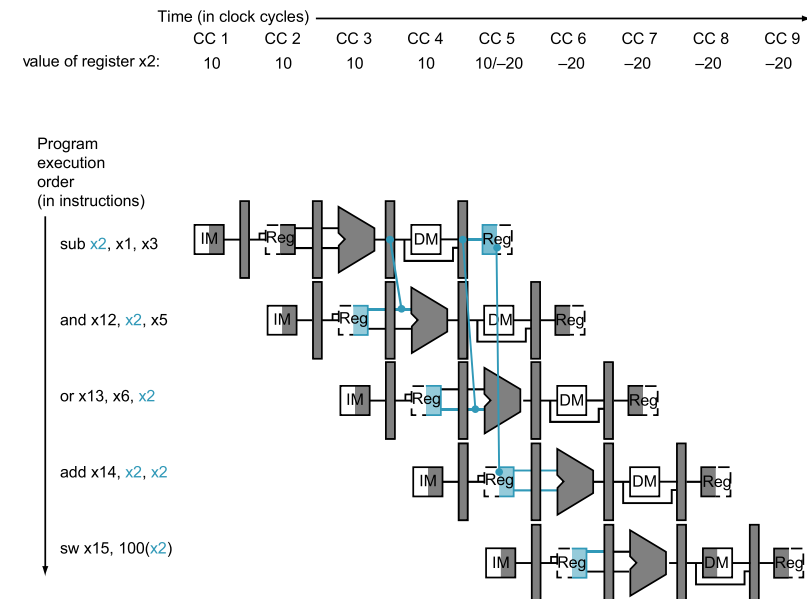
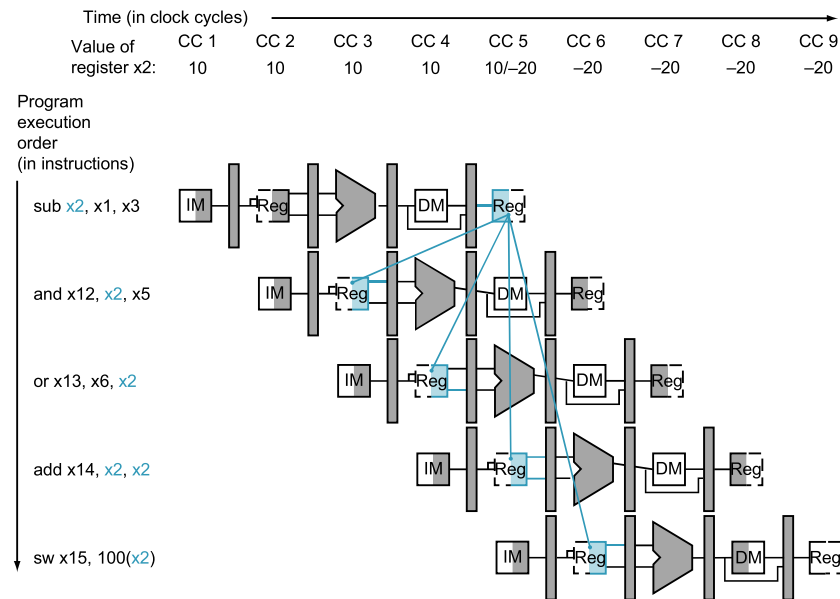


Detecting Data Hazards

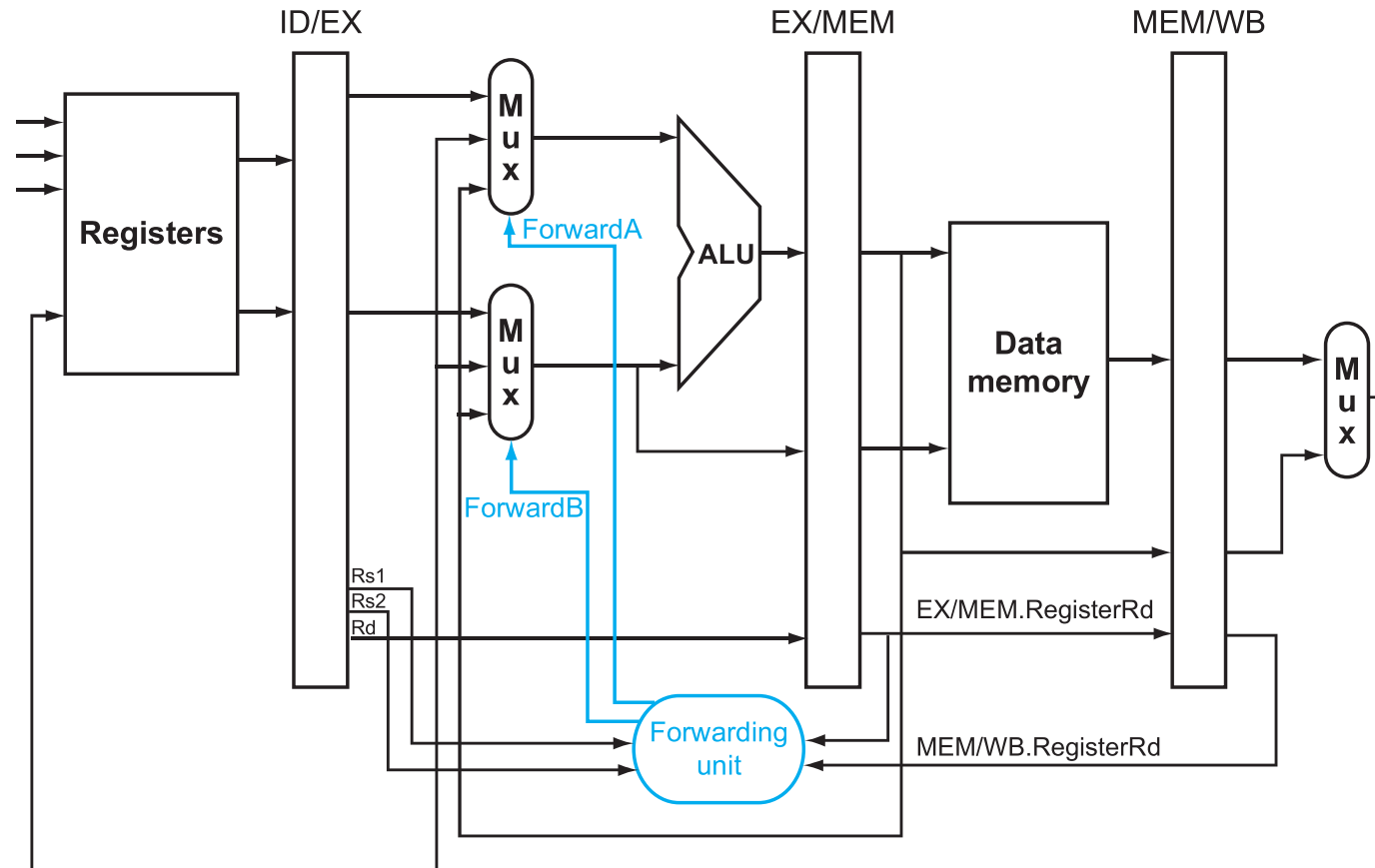
- Final Hazard Detection Policy / Logic
 - EX Hazard
 - $\text{EX/MEM.RegisterRd} = \text{ID/EX.RegisterRs1/Rs2}$
 - $\text{And EX/MEM.RegWrite} = 1$
 - $\text{And Rd} \neq \text{x0}$
 - MEM Hazard
 - $\text{MEM/WB.RegisterRd} = \text{ID/EX.RegisterRs1/Rs2}$
 - $\text{And MEM/WB.RegWrite} = 1$
 - $\text{And Rd} \neq \text{x0}$
 - And there is **NO EX Hazard !!!**
 - **i.e., NOT of EX Hazard conditions!**

Forwarding to Resolve Hazards

- Before and after forwarding ...



Additional Datapath for Forwarding



- Rs1 and Rs2 (addresses) are added in ID/EX register to support forwarding!
- Mux for immediate/register before ALU is not shown!

Forwarding Control

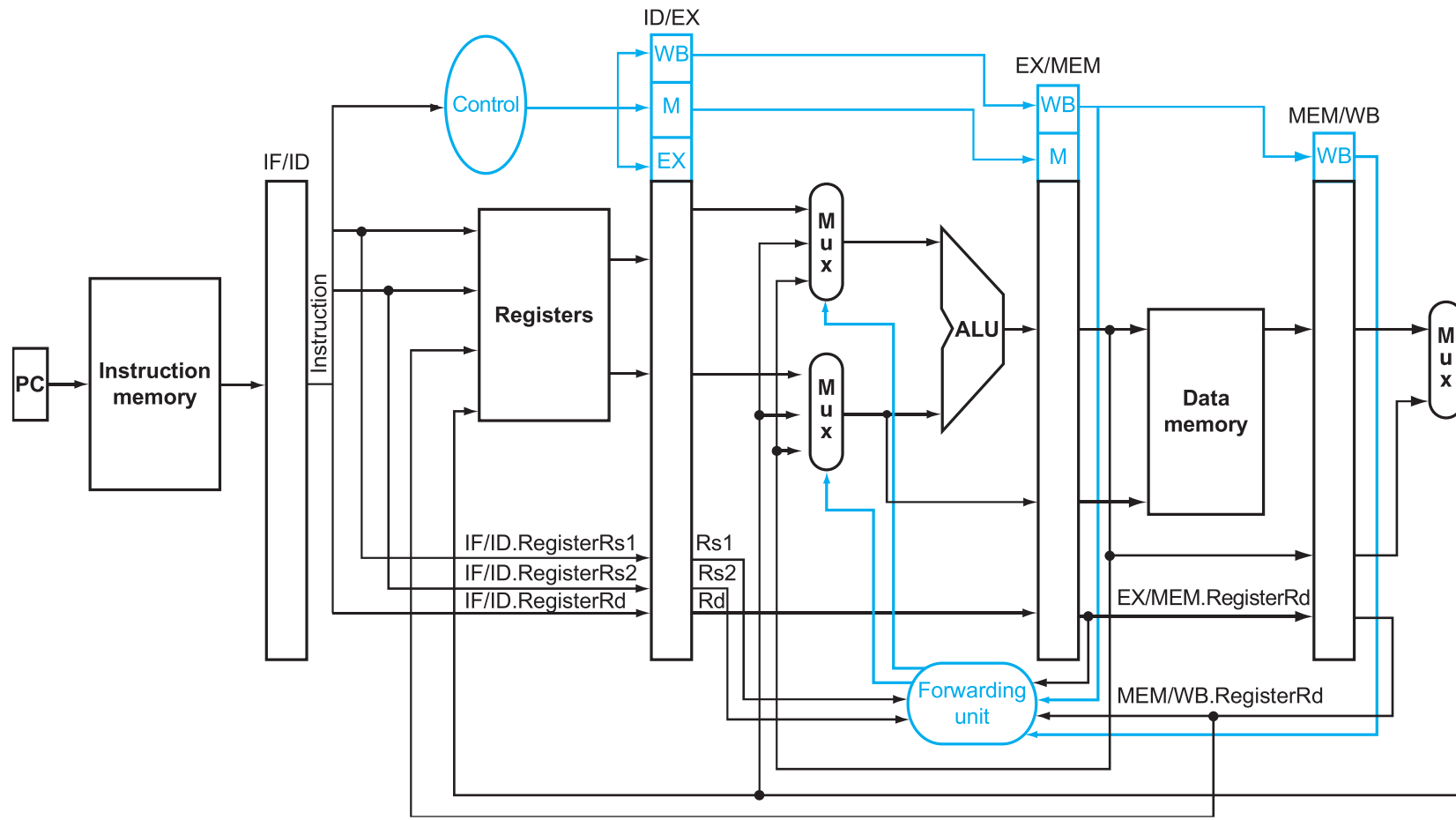
- **ForwardA** and **ForwardB** are the control signals generated by forwarding unit!

Mux control	Source	Explanation
ForwardA = 00	ID/EX	The first ALU operand comes from the register file.
ForwardA = 10	EX/MEM	The first ALU operand is forwarded from the prior ALU result.
ForwardA = 01	MEM/WB	The first ALU operand is forwarded from data memory or an earlier ALU result.
ForwardB = 00	ID/EX	The second ALU operand comes from the register file.
ForwardB = 10	EX/MEM	The second ALU operand is forwarded from the prior ALU result.
ForwardB = 01	MEM/WB	The second ALU operand is forwarded from data memory or an earlier ALU result.

Forwarding Control

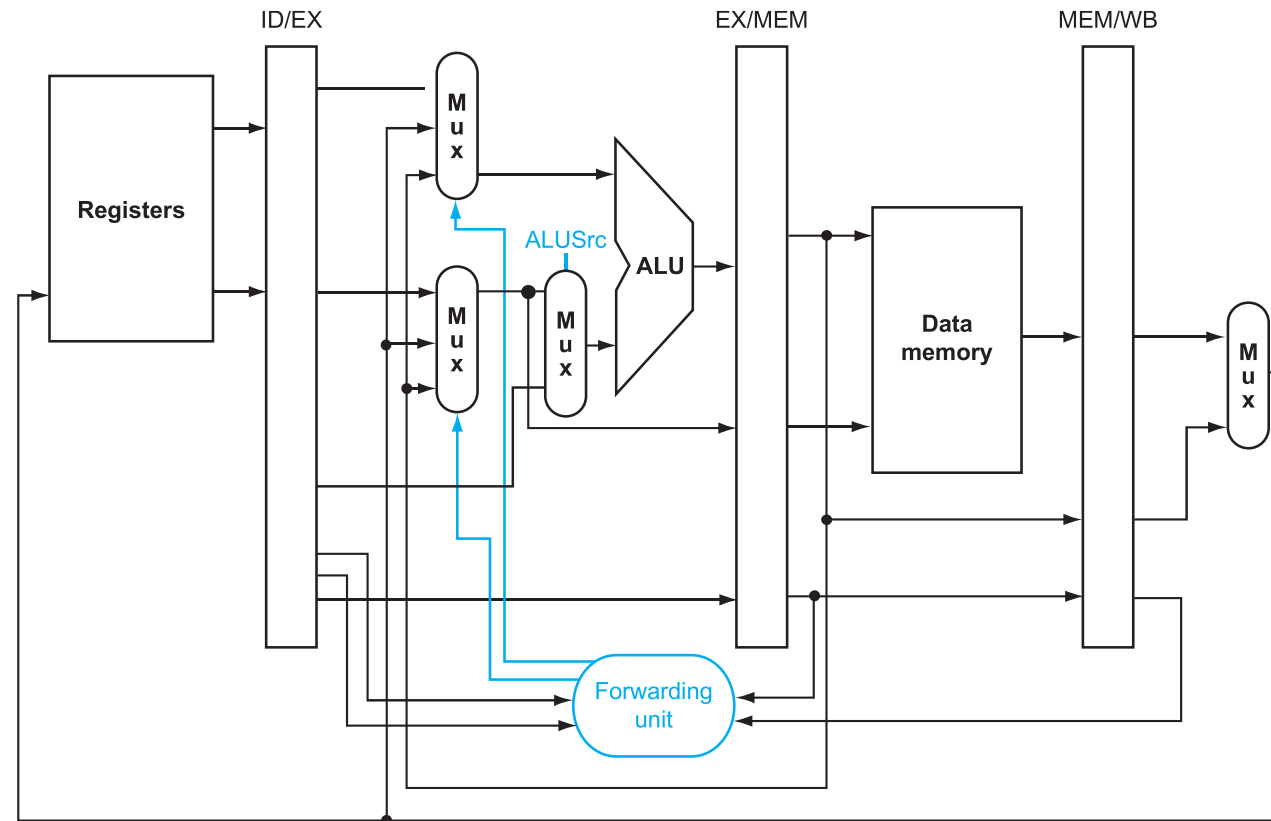
- Logic to implement control in forwarding unit
- EX Hazard
 - if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 10
 - if (EX/MEM.RegWrite
and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 10
- MEM Hazard
 - if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd $\neq$ 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs1))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs1)) ForwardA = 01
 - if (MEM/WB.RegWrite
and (MEM/WB.RegisterRd $\neq$ 0)
and not(EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs2))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs2)) ForwardB = 01

Datapath with Forwarding



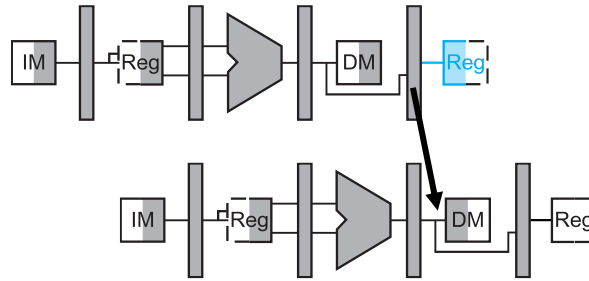
Datapath with Forwarding

- Do not forget to include **ALU Source Mux** ...



The other case of forwarding ...

- What about store following a load?
 - lw x1, 0(x2)
 - sw x1, 0(x3)



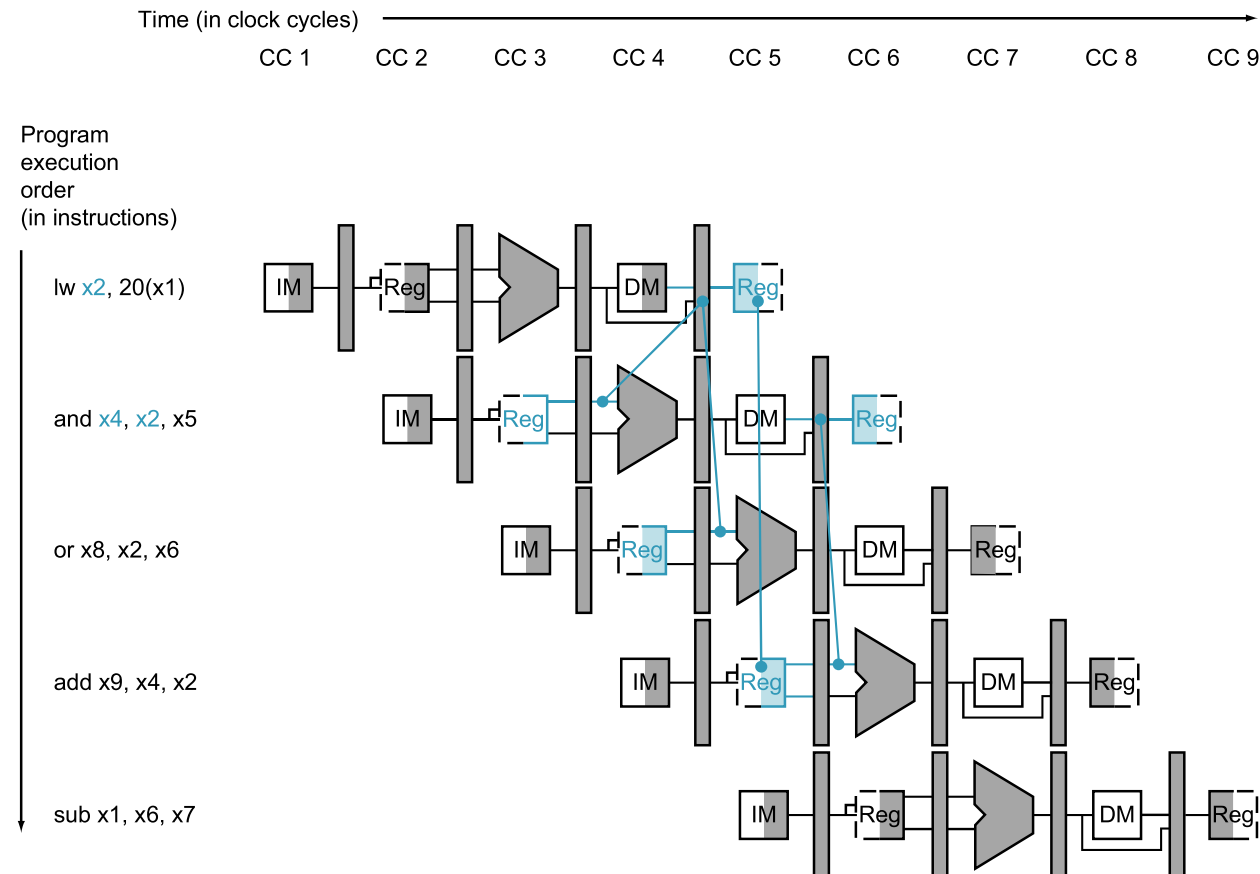
- Need to forward from MEM/WB register to memory access stage! (not to ALU!)
 - Need additional hardware!
 - Try implementing it as a learning exercise!

When Stalls are unavoidable ...



Situations where **we must Stall!**

- When a register is read following a load instruction for the same register ...



**Let's see how we can handle Stalls in
the hardware ...**

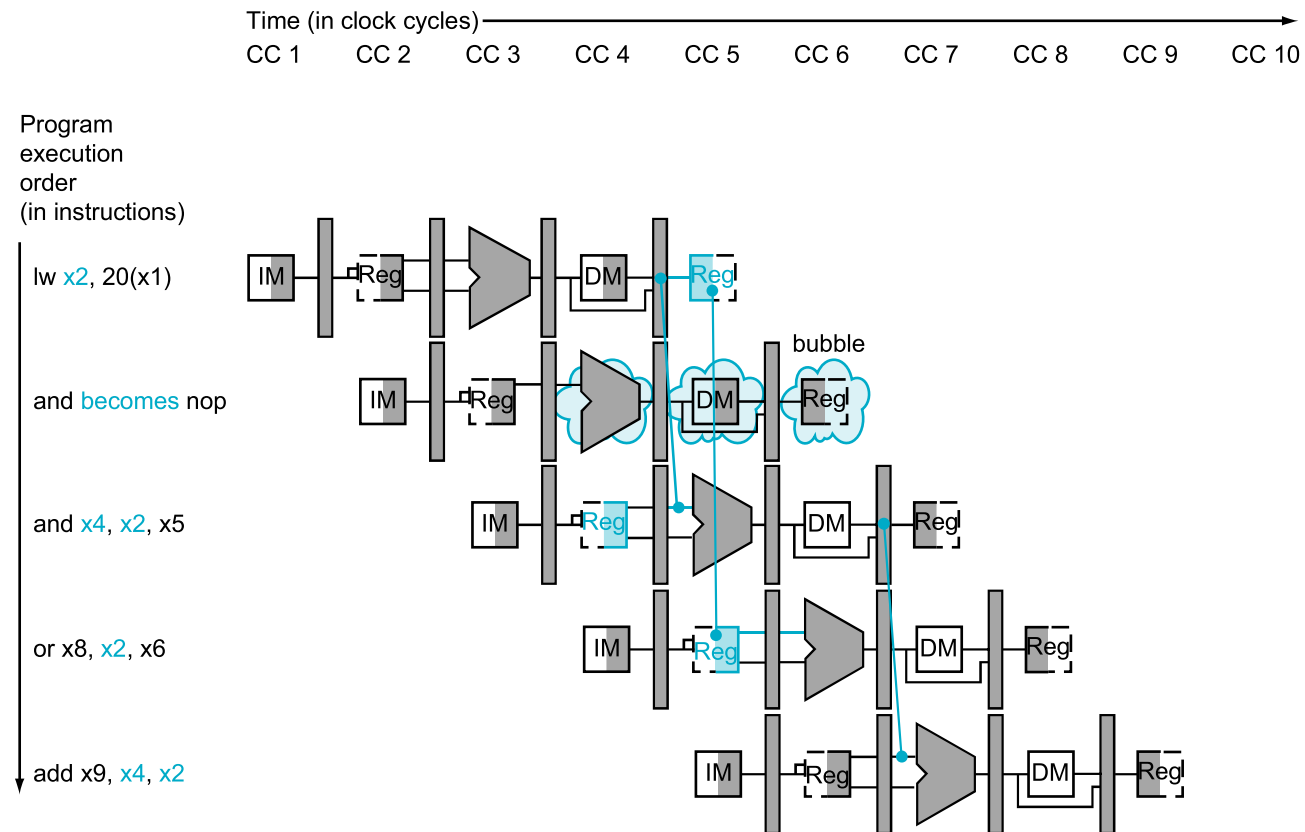


Stalls in Hardware

- Hazard detection unit in the ID stage decides if instruction must be stalled because of the previous load instruction!
 - What stall really is?
 - Can we implement stall by allowing the instruction to execute twice? Would there be any problem?
 - To implement the stall
 - We must not allow the current instruction to progress
 - i.e., do not let the IF/ID pipeline register change!
 - Let the next instruction wait before being fetched into IF/ID!
 - i.e., do not let the PC to change!
 - To prevent the current decoded instruction from execution
 - Do not let register file and memory be written!

Stalls in Hardware

- Instructions execution with a stall!



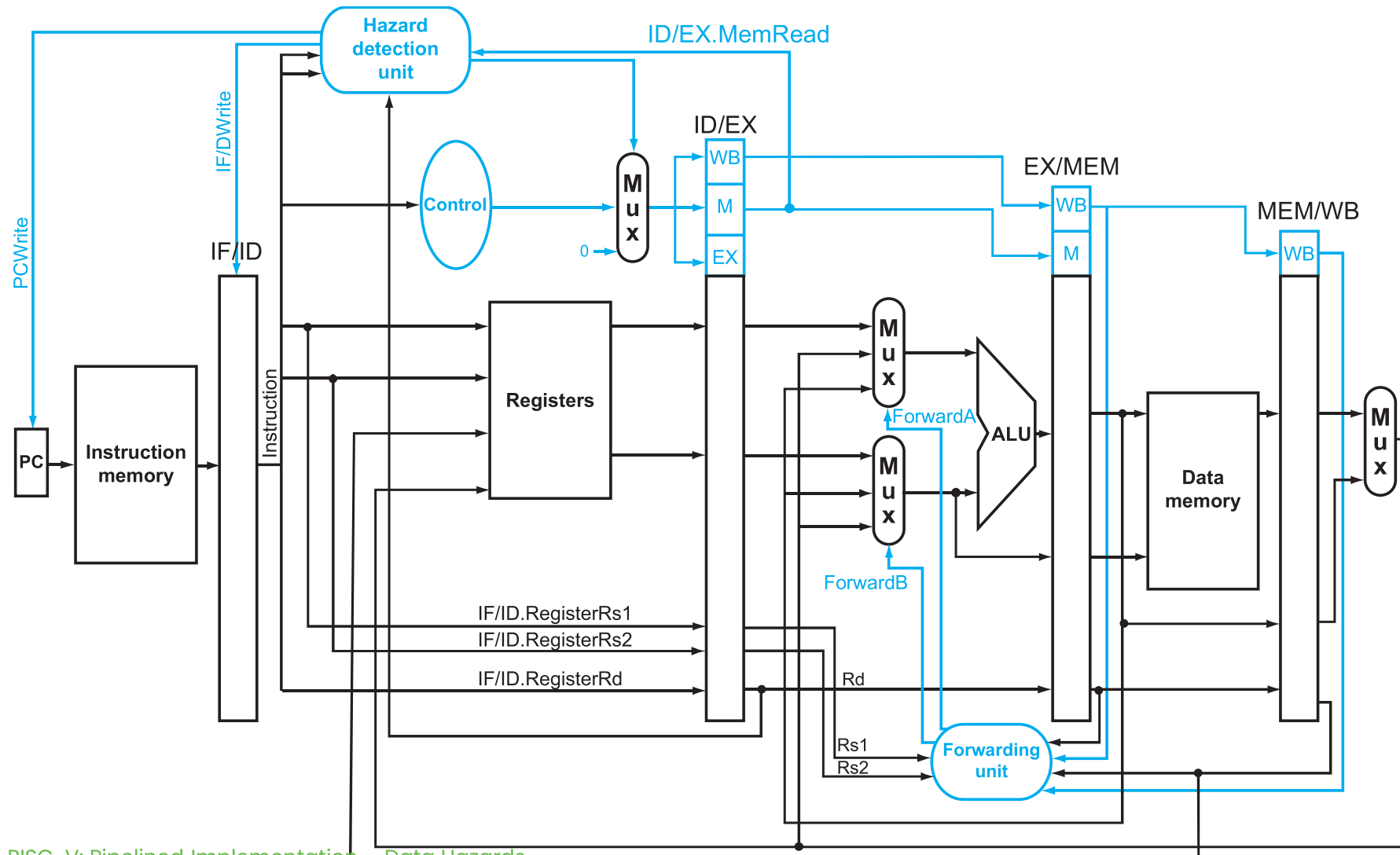
Hazard Detection

- Logic to detect whether to stall an instruction!
 - Check if the previous instruction is a load!
 - Check if the current instruction reads the register loaded by previous instruction!
 - if (ID/EX.MemRead and
((ID/EX.RegisterRd = IF/ID.RegisterRs1) or
(ID/EX.RegisterRd = IF/ID.RegisterRs2)))
stall the pipeline

Control Signals to Implement Stall

- To stall current instruction
 - Do not allow IF/ID register to be changed
 - Use IF/IDWrite signal (write enable signal) to control when IF/ID register should be allowed to be changed!
 - Make the control signals for following stages (Execute, Memory and Write Back) to be 0!
 - Only RegWrite and MemWrite need to be 0, rest can be don't care!
 - Will prevent memory / register file to be written!
 - Practically same as inserting *nop instruction*!
- Preventing next instruction to be fetched
 - Use PCWrite signal (write enable signal) to prevent PC from being updated!

Datapath and Control with Stalls



Knowledge Check!

- Tell whether following code sequences must stall, can avoid stall with only forwarding or can execute without stall or forwarding ...
 - Example 1
 - lw x10, 0(x10)
 - add x11, x10, x10
 - Answer:
 - Cannot fully avoid stall!
 - Can reduce one cycle by forwarding!

Knowledge Check!

- Tell whether following code sequences must stall, can avoid stall with only forwarding or can execute without stall or forwarding ...
 - Example 2
 - add x11, x10, x10
 - addi x12, x10, 5
 - addi x14, x11, 5
 - Answer:
 - Third instruction needs x11 before first writes it back!
 - read-after-write data hazard!
 - Stall can be avoided by forwarding!

Knowledge Check!

- Tell whether following code sequences must stall, can avoid stall with only forwarding or can execute without stall or forwarding ...
 - Example 3
 - `addi x11, x10, 1`
 - `addi x12, x10, 2`
 - `addi x13, x10, 3`
 - `addi x14, x10, 4`
 - `addi x15, x10, 5`
 - Answer:
 - No stalls even without forwarding!

Hardware can fully resolve all data hazards but *code reordering / static scheduling* in software can help prevent some Stalls!

Thank You

